Fly GACA — Source-update runbook

SECTION 06-operations-it TYPE runbook STATUS active OWNER Founder UPDATED 2026-08-19

How Fly GACA keeps the **GACAR**, the **AIP** and the rest of the GACA corpus in sync with GACA's own publications, on the AIRAC cadence.

TL;DR

The pipeline is **two halves with a file between them**. Discovery and extraction happen outside the app repo and hand over a normalised `records.json`; the app-side script diffs that bundle against the live indexes and, on request, merges the deltas.

```
# From a clone of ay2m/FlyGACA, with the bundle at sync-input/records.json:
npm run sync:gaca         # dry run — diffs and reports new / changed / unchanged. Never writes.
npm run sync:gaca:apply   # merge metadata-only deltas, then re-normalise the corpus links
```

`sync:gaca` is a **dry run by default**. That is the safety property to remember: you always see the diff before anything touches the corpus.

What's wired

Piece	Path (in `ay2m/FlyGACA`)	Role
Source manifest	`public/data/sources.json`	The official GACA URLs, the AIRAC config, and the per-source fingerprints. The pipeline owns the fingerprints — don't hand-edit them.
Ingest / diff engine	`scripts/sync-gaca.mjs`	Reads a `records.json` bundle, diffs it against the live indexes, reports, and (with `--apply`) merges.
Merge rules	`scripts/lib/sync-merge.mjs`	Which record kind lands in which index, how a record is keyed, and what counts as "changed".
Link normaliser	`scripts/normalize-corpus-data.mjs` (`npm run data:normalize`)	Rewrites corpus links into the semantic shapes the app routes on. Wired into `sync:gaca:apply`.
Freshness file	`public/data/source-status.json`	Public snapshot — last checked, current/next AIRAC, per-source status. Safe to ship; the app can show "GACAR current as of …".

The record kinds and where they land: **part** → `gacar-index.json` · **ac** → `reference-index.json` · **airport** → `airports.json` · **airspace** and **chart** → their own indexes. All of it under `public/data/`, fetched at runtime — the corpus never enters the JS bundle.

Sources tracked (the "everything from GACA" set)

id	What	URL
`gacar`	GACAR Parts & framework	https://gaca.gov.sa/en/Rules-and-Regulations-Category
`aip` / `aerodromes` / `charts`	AIP / eAIP, AD 2, VFR charts (AIRAC-bound)	https://gaca.gov.sa/web/en/aeronautical-information
`advisory-circulars`	AC-xxx	https://gaca.gov.sa/web/en/advisory-circulars
`notam`	NOTAM / PIB (time-sensitive)	https://gaca.gov.sa/web/en/notam

The AIRAC cadence

AIP currency follows the 28-day AIRAC cycle. The current and next effective dates are computed from the anchor in `sources.json` (`airac.anchor`, `airac.cycleDays`). AIRAC-bound sources stamp their index with the **effective AIRAC date**, not the run date, so the published date matches the cycle.

How a run flows

1. The discovery/extraction half fetches each enabled GACA source and emits a normalised `records.json` bundle (`sync-input/records.json`), each record carrying its `sourceUrl`, `revision` / `effectiveDate` and a `contentHash` fingerprint.
2. `npm run sync:gaca` diffs those records against the live indexes and prints what is new, changed and unchanged — per collection.
3. `npm run sync:gaca:apply` merges the **metadata-only** deltas (new index entries and revisions), persisting each record's provenance fingerprint so the next diff can detect the next revision, then re-runs the link normaliser.
4. **Body-bearing records** — anything carrying a raw PDF or eAIP asset — are reported as **deferred**, not written. Turning a refreshed source document into per-Part reading content is the conversion step, and it is deliberately not automatic.
5. Downstream rebuilds after a corpus change: `npm run build:chunks` (Captain Adel's retrieval index) and `npm run parse:regulations` if the cross-reference lookup is affected. Then `npm run verify` before committing.

Fail-safe behaviour

- **The default is a dry run.** Nothing is written without `--apply`.

- **The synthetic sample fixture cannot be applied.** If the only bundle present is `records.sample.json`, `--apply` refuses outright rather than writing `[SAMPLE]` records into the real corpus.
- **Merges are additive and idempotent.** Re-running with the same bundle is a no-op; a record whose fingerprint hasn't moved is reported unchanged.
- **The normaliser is lossless.** Any link it can't resolve keeps its original string and is reported, so nothing is silently dropped.

Scheduling

There is **no scheduled workflow doing this today** — `ay2m/FlyGACA` ships without a `.github/workflows/` directory, so CI (and any cron) has to be wired against your own project first. In practice the cadence is: run the discovery half, then `npm run sync:gaca` around AIRAC Thursdays and after any GACA publication you hear about, review the diff, and apply.

If you do automate it, keep the same two-step shape — a scheduled job that only ever runs the dry run and reports, with the apply left to a human, is the version that cannot quietly corrupt the corpus.

> [!NOTE] Earlier versions of this page described `npm run check:sources` / `npm run update:sources`, a `scripts/update-sources.js` scraper, a `.github/workflows/update-sources.yml` cron, a `check-data.js` integrity gate, a `library/` staging tree, `assets/data/` paths, and a `FIREBASE_TOKEN` secret for auto-deploy. **None of those exist.** The commands above are the real ones.

Operational runbook — not legal advice. GACA remains the authoritative source.